Plant Doctor: A Context-Aware Multimodal AI Agent

Mohit Aggarwal
Data and Systems
Yours Truly Inc.
Richmond, USA
maggarwal.ytihomecare@gmail.com

Kanika Sood
Department of Computer Science
California State University, Fullerton
Fullerton, USA
kasood@fullerton.edu

John Morkovci
Department of Computer Science
California State University, Fullerton
Fullerton, USA
jovci@csu.fullerton.edu

***Abstract*—Home gardeners often face recurring pest damage, while existing digital tools offer only narrow, image-based identification without contextualized treatment guidance. In our backyard case study, a multimodal large language model correctly identified a Swallowtail caterpillar from a photo but failed to account for local weather, soil conditions, or product availability, making its advice impractical. To address this limitation, this paper presents the Plant Doctor, an AI agent that combines multimodal perception with hyper-local environmental data and product search to generate actionable treatment plans for garden pests. The system utilizes Google's Gemini 2.5 Flash as an orchestrator, integrates the National Weather Service and USDA soil APIs, and connects to an Amazon catalog of organic treatments through a two-step prompting strategy that separates expert guidance from product retrieval. Exposed via the Model Context Protocol and deployed as a containerized web service, the Plant Doctor illustrates how agentic AI systems can move beyond generic answers toward practical, context-aware assistance.**

***Index Terms*—AI Agents, multimodal large language models, context-aware recommendation, home gardening, agricultural decision support, Model Context Protocol**

I. Introduction

Home gardening environments are prone to recurring issues such as diseases, improper care, and pests, which can significantly reduce yields. In our garden, a small lemon tree developed noticeable damage caused by a caterpillar. A multimodal large language model was used to analyze an image of the pest and correctly identify it as a Swallowtail caterpillar. Although the response included a treatment suggestion, it did not consider the ongoing dry spell in the area, the garden's sandy soil, or the availability of organic products that could help the lemon tree.

This scenario illustrates the failure of AI systems: they provide technically correct but shallow responses that ignore crucial context. While pest identification is a necessary first step, it is not sufficient on its own. Effective systems must provide treatment guidance in the context of a user's local conditions. In this setting, what is required is not only an identifier but also an effective "plant doctor" that can integrate visual perception with localized environmental data and product availability to recommend a proper course of action.

Motivated by this observation, the present work develops the Plant Doctor, a full-fledged AI agent that transforms a single plant image and location into a comprehensive context-aware treatment plan. The system is designed to reason across multiple specialized sources and to act as a consultant rather than a static encyclopedia. It combines visual, weather, soil, and product data to deliver recommendations that align with the user's environment and constraints. Prior work on multi-agent and agentic LLM systems demonstrates that structured tool use can improve reliability and factual consistency [1], [4], [8], [10]. We build on these ideas and adapt them to the practical setting of home gardening management.

The main contributions of this work are:

- Design of a multimodal, context-aware AI agent for home gardening management that combines image-based diagnosis with local weather and soil information.
- Introduction of a two-step prompting workflow that separates treatment plan generation from downstream product matching.
- Construction of a specialist toolkit that integrates USDA and National Weather Service Soil Data APIs, together with an Amazon product search, to support actionable recommendations.
- Implementation of a Model Context Protocol interface and a containerized, continuously deployed cloud architecture, enabling reuse by external AI clients.

II. Related Work

Recent work on multimodal and agentic AI systems has explored how large language models can integrate visual perception with external tools and structured reasoning to support real-world tasks. These efforts focus on applications that extend from image understanding to interactive decision-making within local context.

Several studies investigate general-purpose multimodal agents that combine vision, language, and tool use to act in complex environments. These systems demonstrate how visual inputs can be interpreted and mapped to reasoning steps, often through coordinated layers that select and invoke specialized tools [14]. While such approaches show strong performance on benchmarked tasks and simulated environments, they typically emphasize broad capability demonstrations rather than domain-specific decision support.

More recent work has begun to emphasize agentic pipelines that separate perception, reasoning, and action. In the agricultural domain, MA3 proposes a multimodal agricultural agent architecture that routes image-based disease queries to specialized tools for classification, detection, and visual question answering [15]. While this approach demonstrates strong vision-centric disease analysis, it does not incorporate external environmental context such as soil composition or weather-dependent treatment timing.

Other multimodal systems focus specifically on plant disease diagnosis. PDD-AGENT introduces a multimodal large language model–driven agent that combines visual inputs with natural language reasoning to improve plant disease identification and explanation [16]. Although this work advances interpretability and diagnostics, its scope is primarily limited to disease recognition and explanation, without integrating hyper-local environmental data or translating diagnoses into treatment plans.

Comprehensive reviews of AI-based plant disease detection methods show that most existing systems emphasize image acquisition, preprocessing, and classification accuracy, evaluated in controlled or laboratory-style settings [17]. These studies note that limited incorporation of environmental context and lack of decision-support integration remain significant barriers to real-world deployment.

In contrast to prior multimodal and agricultural agent systems, the Plant Doctor focuses on a specific domain: home gardening and plant health management. Rather than treating images as isolated inputs, the system explicitly integrates visual diagnosis with hyper-local weather forecasts, soil-profile data, and product availability to produce context-aware treatment guidance. By linking perception, environmental constraints, and downstream actions within a agentic workflow, the Plant Doctor acts as a full decision-support system for real-world plant care.

III. System Design and Architecture

A. From Identification to Recommendation

Initial experiments with off-the-shelf image-to-text systems demonstrated that one-step pest identification is insufficient for gardening support. To be truly useful, an assistant must transition from classification to informed recommendation. The target behavior is an intelligent agent that can select and invoke appropriate tools to construct an end-to-end answer, rather than rely on a single approach.

The Plant Doctor is designed as an agricultural assistant capable of:

1) Interpreting user-provided images of plants and pests.
2) Incorporating hyper-local weather and soil conditions.
3) Mapping soil and weather advice to organic treatment products.
4) Producing a user-ready treatment plan in a single interaction.

B. Multimodal Specialist Toolkit

To support these objectives, the Plant Doctor equips its intelligent agent with a toolkit consisting of four specialized components:

1) **The Eyes (Gemini 2.5 Flash):** A multimodal model that processes uploaded photographs to identify the pest, the host plant, and the severity of the infestation [2], [5].
2) **The Local Expert (Government APIs):** A set of calls to the National Weather Service for real-time local forecasts and to the USDA Soil Data Access API for detailed soil composition data in the user's zip code.
3) **The Pharmacist (Product Search Tool):** A product search interface that locates specific organic treatment products available on Amazon and returns direct purchase links to make recommendations immediately actionable.
4) **The Universal Adapter (MCP Interface):** A standardized protocol layer that exposes the toolkit functions so that other AI clients can invoke the Plant Doctor's specialized capabilities as modular services.

By combining these components, the Plant Doctor behaves like a team of specialists coordinated by a central orchestrator. The orchestrator, implemented using Gemini 2.5 Flash, determines which tools to call and in what sequence, before unifying the results into a personalized response (Figure 1).

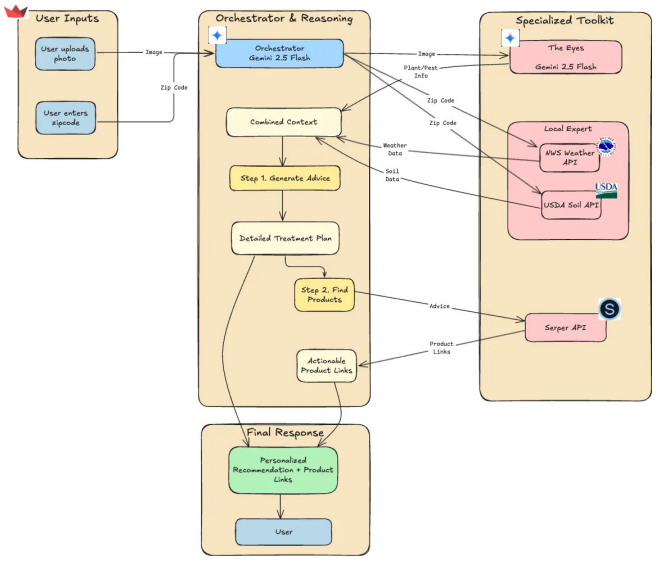


Fig. 1. High-level architecture of the Plant Doctor agent, including user inputs, the Gemini orchestrator, the specialist toolkit, and the two-step reasoning process that produces a personalized treatment recommendation.

C. Implementation Challenges

The transition from concept to fully deployed application required addressing several challenges. These challenges shaped the final design of the Plant Doctor and served as a proving ground for its architecture.

Early versions of the agent produced incomplete outputs. For instance, the system occasionally returned a list of organic

products without providing the underlying treatment advice, effectively acting as a prescription generator without a diagnosis. This behavior indicated that the agent was being tasked with too much in a single step, leading to skipped reasoning and hallucinated structure.

Initially the system exhibited deployment fragility. Code that operated correctly on a local Apple laptop failed when transferred to a cloud server due to differences in the execution environment. The application had been tied to a specific platform, causing compatibility issues when moved to a Linux-based infrastructure.

The original user experience required users to navigate across multiple screens. After uploading a photo and receiving an assessment, users were redirected to a separate interface to enter location information, losing sight of the original plant image and the associated problem. This design was confusing and disrupted the task flow.

These challenges motivated three key design responses: separating the agent's reasoning into sequential steps, containerizing the application with Linux compatibility using Docker, and refactoring the user interface to present the image, assessment, and local data together on a single screen. Collectively, these responses improved the Plant Doctor's reliability, portability, and usability.

D. Reasoning Process and Prompt Design

The Plant Doctor's reasoning process is implemented as a two-step prompt chain. Early attempts relied on prompts that asked the AI system to identify the pest, construct a detailed treatment plan, and find corresponding products within a single interaction. This approach yielded inconsistent behavior, with the agent sometimes skipping steps or hallucinating components.

To address this, the reasoning pipeline was restructured so that the AI is treated as a junior assistant that receives instructions. The process is divided into an advice phase followed by a product-matching phase:

1) **Advice Generation:** The orchestrator is instructed to act as an agricultural expert and to write a detailed, multi-paragraph treatment plan. The prompt explicitly forbids mentioning any specific products in this step.
2) **Product Retrieval:** The system then uses the generated treatment plan as context for a second prompt that instructs the AI to search for matching organic products via the product search tool.

Conceptually, the logic underlying this two-step design proceeds as follows. The system aggregates all available context about the user's situation, including the plant image, the detected pest, the severity of damage, and the local environmental conditions. Based on this information, the agent creates a structured treatment plan that explains the diagnosis, recommended actions, and any relevant precautions.

Only after this plan has been created does the agent transition to the second phase, where it interprets the treatment steps and uses them as constraints for product selection. At this stage, the agent focuses on identifying organic products that align with the treatment plan, are compatible with the user's environmental conditions, and are available for purchase. The final response presented to the user combines both the narrative treatment guidance and the product recommendations in a single, clear answer.

E. Contextual Data Integration

To move beyond generic advice and mimic the behavior of a real-world consultant, the Plant Doctor incorporates hyper-local environmental context. When a user submits a zip code, the agent automatically queries:

- The National Weather Service for local real-time forecasts [3].
- The USDA Soil Data Access API for detailed soil composition in the corresponding area [6].

The location service integrates weather and soil information into a shared context representation, which is passed to the orchestrator along with the plant image and user goal. This representation summarizes the most relevant aspects of the local environment, such as temperature range, recent precipitation patterns, and soil type.

By incorporating this context into the reasoning process, the agent can justify its recommendations in terms of both what is visible in the image and what is known about the plant's environment. For instance, it can recommend adjusting watering practices to compensate for an ongoing dry spell or recommend soil improvements that are compatible with the detected soil composition. The resulting context is then passed to the orchestrator along with the plant image, ensuring that treatment recommendations contain both visual and environmental information.

F. Interoperability and Deployment

To position the Plant Doctor as more than a standalone application, its internal tools are wrapped in a Model Context Protocol (MCP) format [7], [9]. Functions such as *get_weather* and *get_soil_data* are exposed through an interface that can be consumed by any MCP-compatible client. This design prevents the system from becoming isolated and instead transforms it into a modular, reusable service. External AI agents from different providers, including future models, can call the Plant Doctor's capabilities as part of larger multi-agent workflows.

Within the MCP framework, each capability is presented as a tool, followed by a clear description, input schema, and output schema. The weather tool, for example, accepts a location identifier and returns a structured summary of current and upcoming conditions relevant to treatment decisions. Our soil tool exposes similar semantics for soil composition, while the product-search tool receives a textual description of treatment needs and returns a list of recommended products. By standardizing these tools, the Plant Doctor can evolve independently.

To ensure portability and scalability, the Plant Doctor is packaged within a Docker container, which is configured for Linux compatibility. This decision resolves the initial "works on my machine" issue by decoupling the application from

the local development environment and aligning it with the cloud runtime. The containerized application is deployed on Google Cloud Run, a serverless platform that automatically manages scaling behavior. As user traffic fluctuates, Cloud Run automatically adjusts the underlying resources to ensure the application remains responsive, eliminating the need for manual infrastructure management.

To streamline updates, the system utilizes an automated pipeline based on GitHub Actions. When changes are pushed to the main code branch, a workflow is triggered that builds a new container image, runs basic tests, and, if successful, deploys the updated image to Cloud Run. This continuous delivery pipeline helps transform manual deployment steps into an automated, reliable process. Developers can focus on enhancing the Plant Doctor's capabilities, prompts, and user experience, while the pipeline ensures that verified changes are deployed to production in a consistent and repeatable manner.

We measured end-to-end latency and estimated API cost per query to document deployment characteristics. Table I summarizes the averages across 16 test queries.

TABLE I
AVERAGE BENCHMARK METRICS PER QUERY

Metric	Average Value
Input Tokens	1,293
Output Tokens	151
End-to-End Latency	4.38 s
Estimated API Cost	$0.00077

IV. USE CASES

A. Swallowtail Caterpillar on Lemon Tree

In this use case, we uploaded a photograph (Fig. 2) of a lemon tree leaf with a caterpillar, along with its corresponding zip code. Plant Doctor identifies the pest as a Swallowtail caterpillar based on the uploaded image and recognizes the host plant as a citrus tree.



Fig. 2. Photograph of a citrus leaf showing a Swallowtail caterpillar, submitted to the Plant Doctor for diagnosis.

Using the provided location, the system retrieves local weather forecasts and soil information. At the time of the query, the Plant Doctor classifies the case as a Swallowtail caterpillar on a citrus leaf and warns that these feeders can cause significant Defoliation and stress the plant, reducing yield. The generated treatment plan prioritizes removal of damaged leaves and manual control of visible caterpillars, while recommending organic treatment options that minimize stress under local conditions. The suggested products are selected to align with the proposed control methods and are confirmed to be readily available through a major e-commerce platform, ensuring the recommendations are immediately actionable (Fig. 3).

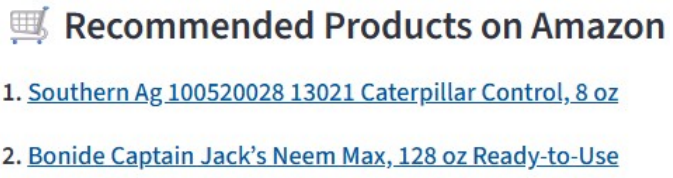


Fig. 3. Screenshot of the Plant Doctor interface showing product treatment recommendations.

This initial use case demonstrates the Plant Doctor's ability to complete an end-to-end workflow, from image-based diagnosis through expert reasoning to product-level intervention.

B. Hibiscus Leaf Spot with Weather-Aware Timing

For the second use case, we evaluated the Plant Doctor on a photograph of a hibiscus leaf showing multiple dark, necrotic lesions (Fig. 4). Originally documented as a leaf-spot case on hibiscus [11], was used to test whether the agent could generate disease management advice.



Fig. 4. Hibiscus leaf exhibiting dark, necrotic lesions used as input for the hibiscus leaf-spot use case (image reproduced from Nelson [11]).

Supplying the hibiscus photograph together with a zip code in Fairfax, CA (Fig. 5). The Plant Doctor classifies the issue as leaf spot on hibiscus with moderate severity and warns that unchecked infections can lead to leaf drop and reduced photosynthesis, weakening the plant. The generated treatment plan recommends removing affected leaves and debris around the base of the plant, improving air circulation through selective pruning, and avoiding overhead watering, since wet foliage could lead to further disease spread.

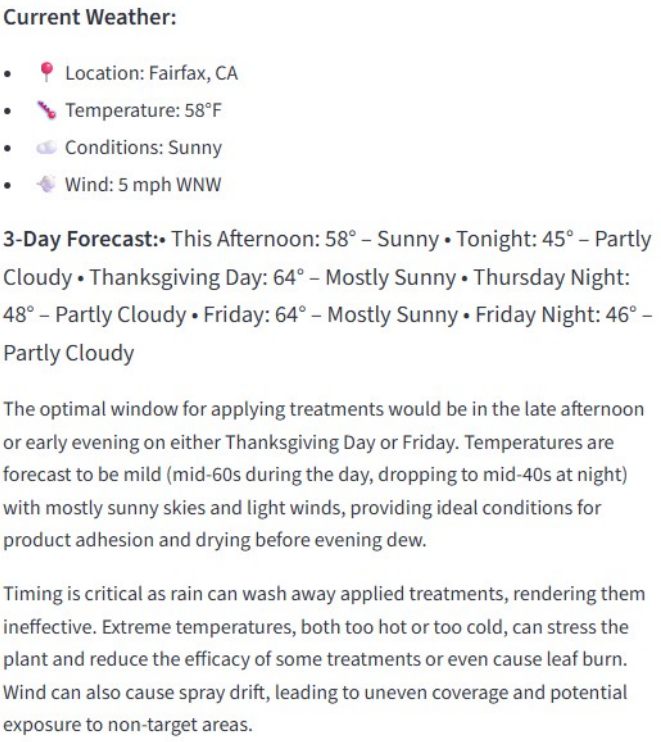


Fig. 5. Weather-timing panel from the Plant Doctor interface for Fairfax, CA, summarizing current conditions, a three-day forecast, and narrative guidance on the optimal treatment window for the hibiscus leaf-spot use case.

Because the soil data for the given coordinates are incomplete, the system explicitly notes this limitation and instead provides general soil-handling guidance, such as maintaining good drainage and avoiding waterlogging at the roots. The weather-timing view then queries the National Weather Service and summarizes the current conditions and a three-day forecast for Fairfax. In the captured run, the forecast is sunny with light winds and mild temperatures, so the agent recommends applying any foliar treatments in the morning on upcoming dry days, after the dew has evaporated, to maximize adherence and minimize the risk of moisture on leaves.

This case illustrates how the Plant Doctor uses hyperlocal weather data to refine treatment timing even when soil information is unavailable. The agent directly accounts for this uncertainty and compensates by prioritizing weather-aware application windows and conservative cultural practices, demonstrating decision-making under an incomplete environmental context.

C. Aphids and Ant Presence on Taro Leaf

A third use case evaluates the Plant Doctor on a photograph of a taro leaf heavily infested with aphids and ants (Fig. 6). Originally documented as an aphid and ant association on taro in Mānoa Valley, Honolulu [12], was used to test the agent's ability to reason about both pest pressure and local soil conditions.



Fig. 6. Taro leaf with dense aphid infestation and attendant ants used as input for the Colocasia aphid use case (image reproduced from Ser Amantio di Nicolao [12]).

Given the infected leaf (Fig. 6) and a Honolulu zip code, the Plant Doctor classifies the issue as severe aphid feeding worsened by ant presence and states that untreated damage can rapidly weaken the plant, stunting growth and reducing yield. The treatment plan recommends a multi-pronged strategy: physically removing as many aphids as possible, disrupting the protective role of ants around the colonies, and applying organic foliar treatments such as insecticidal soap or neem-based sprays. Beyond initial intervention, the system emphasizes ongoing monitoring as a critical component of control. It advises inspecting plants every few days, particularly the undersides of leaves and new growth, to assess reductions in aphid and ant populations and to detect early signs of reinfestation.

The soil-impact view (Fig. 7) queries the USDA Soil Data Access API and reports that the site is mapped as Makiki clay

loam with 0–2% slopes, a well-drained clay loam containing roughly 31.5% sand, 37.5% clay, and 31.0% silt, with pH 5.3 and organic matter around 8.5%. The agent explains that this soil provides a fertile, moisture-retentive but well-drained medium that generally favors healthy taro growth, while the high organic matter further supports vigor despite the acidic reaction.

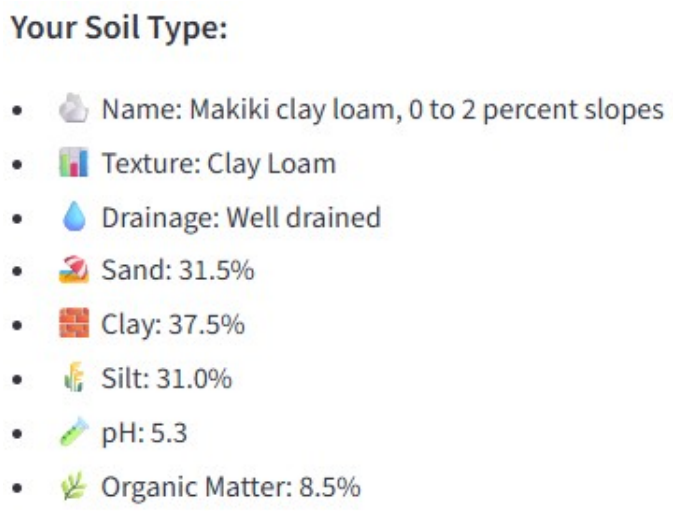


Fig. 7. Soil-profile summary returned by the Plant Doctor for a Honolulu zip code, indicating Makiki clay loam with well-drained texture, slightly acidic pH, and relatively high organic-matter content.

The Plant Doctor then links these properties to treatment strategy. Because the Makiki clay loam drains well, the system advises careful timing and even distribution of any soil-applied products to avoid leaching and to maximize root uptake. Foliar treatments are recommended for early morning or late afternoon under Honolulu's warm, sunny, and often hazy conditions, reducing evaporation and the risk of leaf burn. The provided response and information highlights the agent's ability to couple pest diagnosis with soil profiles, adapting both treatment choice and application timing for an ideal garden.

D. Anthracnose on Mango with Soil- and Weather-Aware Timing

In the final use case, the Plant Doctor is evaluated on a photograph of a mango fruit covered with dark, sunken lesions consistent with anthracnose (Fig. 8).



Fig. 8. Mango fruit with dark, sunken lesions used as input for the anthracnose-on-mango use case (image reproduced from Wikimedia Commons [13]).

Given our mango (Fig. 8) and a zip code in West Palm Beach, FL, the agent classifies the issue as severe anthracnose on mango and states that unmanaged infections can cause fruit rot and major yield loss. The generated treatment plan focuses first on canopy sanitation and cultural measures: removing infected fruit and shoots, thinning interior branches to improve airflow, and maintaining strict hygiene by disposing of fallen debris. Once these steps are in place, the system recommends a protective fungicide spray during high–risk periods, following label directions and visual monitoring of new flushes and fruit.

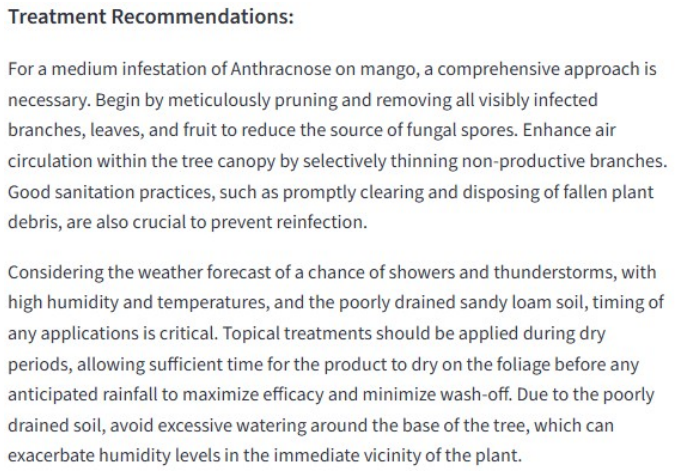


Fig. 9. Comprehensive treatment report generated by the Plant Doctor for the anthracnose-on-mango use case, summarizing pruning and sanitation steps, soil-impact analysis, weather-based timing, and recommended fungicidal products.

The soil–impact and weather–timing views set how and when those treatments (Fig. 9) should be applied. USDA data indicate that the local soil is mapped as Basinger fine sand, a poorly drained sandy loam with very high sand content, low organic matter, and an acidic pH. The Plant Doctor explains that this combination favors persistent surface moisture and

humidity around the lower canopy, conditions that promote anthracnose while also limiting nutrient availability and tree health. In parallel, the weather forecast shows repeated chances of showers and thunderstorms followed by a drier, mostly sunny day, so the agent advises delaying fungicide applications until the dry window, ensuring foliage is dry at spraying time, and no significant rain is expected for 24–48 hours to prevent wash-off maximizing product adhesion. This scenario underscores how the Plant Doctor links disease diagnosis with site-specific soil constraints, short-term weather patterns, and ongoing monitoring to manage fungal risk in perennial fruit systems

Among the four test cases, this scenario most clearly shows the Plant Doctor operating as a full decision-support system: it combines image based disease diagnosis with explicit soil-profile data and a multi-day weather forecast to tailor both treatment choice and application timing for ideal crops. Unlike earlier cases that emphasize either weather or soil in isolation, this example requires the agent to balance both factors simultaneously under disease pressure. The result is a recommendation that is not only technically correct but also feasible within the narrow treatment windows imposed by local environmental conditions.

V. Limitations and Future Work

The current implementation of the Plant Doctor has specific limitations that direct our immediate future development. Because the reasoning pipeline operates sequentially, an upstream visual misidentification by Gemini 2.5 Flash will cascade into the treatment and product retrieval steps. Future iterations will incorporate confidence scoring to explicitly surface diagnostic uncertainty to the user.

The product retrieval mechanism depends on standard e-commerce APIs. These interfaces are vulnerable to inventory churn, regional link-rot, and sponsored-product bias. Future versions will validate product availability programmatically before presenting the link to the user.

Measuring the practical efficacy of context-aware recommendations requires quantitative evaluation. We list a formal human-in-the-loop user study comparing recommendations with and without environmental context as our primary target for future work.

The current agent relies on US-centric tools for weather and soil data. The underlying MCP design is modular, allowing developers to swap out the US-centric tools for global alternatives. Adapting the architecture for non-US regions requires exchanging these sources for tools such as OpenWeatherMap for global forecasts or the Food and Agriculture Organization (FAO) digital soil map.

The current implementation relies on the Google Generative AI software development kit, creating a strict dependency on the Gemini model family. Switching to alternative Gemini models requires only minor configuration changes. Transitioning to models from other providers requires rewriting the image payload format, function-calling schemas, and chat session state management. We maintain this architecture to prioritize reliable tool execution, acknowledging that this dependency limits the ability to perform ablation studies across different multimodal models.

VI. Conclusion

The Plant Doctor system originated from a gardening challenge and has evolved into a more ambitious demonstration of context-aware AI assistance. By augmenting image-based identification with local weather and soil information, as well as product availability, the system delivers treatment recommendations that are both technically sound and practically useful. The architecture combines a multimodal orchestrator, a specialist toolkit, a two-step prompting strategy, and an MCP-based interoperability layer, all deployed via containerized cloud infrastructure with continuous delivery. This configuration illustrates how intelligent agents can move beyond generic, one-size-fits-all solutions and instead function as collaborative partners in addressing everyday problems. The Plant Doctor provides a blueprint for future systems in which AI agents integrate perceptual inputs and environmental context to support informed decision-making in real-world applications. If an AI system can be constructed to effectively protect a garden, similar design patterns may be applied to a wide range of other domains where context-sensitive assistance is required.

References

[1] Q. Wu, G. Bansal, J. Zhang *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," 2023, arXiv. [Online]. Available: https://arxiv.org/abs/2308.08155

[2] Google, "Gemini API documentation," 2025, Google AI for Developers. [Online]. Available: https://ai.google.dev/gemini-api/docs

[3] National Weather Service, "API web service documentation," 2025, NWS. [Online]. Available: https://www.weather.gov/documentation/services-web-api

[4] M. Aggarwal, "Beyond the prompt: An architect's guide to production-ready AI agents," 2025, Medium. [Online]. Available: https://medium.com/@mohitagr18/beyond-the-prompt-an-architects-guide-to-production-ready-ai-agents-4df49c1c8553

[5] Google Cloud, "Generative AI on Vertex AI: Live API reference," 2025, Google Cloud Docs. [Online]. Available: https://cloud.google.com/vertex-ai/generative-ai/docs/model-reference/multimodal-live

[6] USDA NRCS, "Soil Data Access API — tabular POST endpoint," Soil Data Access (SDA). [Online]. Available: https://sdmdataaccess.nrcs.usda.gov/Tabular/post.rest

[7] FastMCP, "Framework overview and quickstart," 2023–2025. [Online]. Available: https://gofastmcp.com

[8] Streamlit Community, "Build a research agent with LangGraph and Streamlit," 2025. [Online]. Available: https://discuss.streamlit.io/t/build-a-research-agent-with-deepseek-langgraph-and-streamlit/97580

[9] J. Lowin, "FastMCP: Fast, Pythonic MCP servers and clients," GitHub repository, 2025. [Online]. Available: https://github.com/jlowin/fastmcp

[10] M. Aggarwal, "A step-by-step guide to building an agentic RAG agent with LangGraph," 2025, Medium. [Online]. Available: https://medium.com/@mohitagr18/the-ai-that-thinks-before-it-searches-a-deep-dive-into-agentic-rag-82e5db9a0826

[11] S. Nelson, "Hibiscus: Bacterial leaf spot," Wikimedia Commons, CC0 1.0 Universal Public Domain Dedication, Mar. 6, 2018. [Online]. Available: https://commons.wikimedia.org/wiki/File:Hibiscus_bacterial_leaf_spot_(5903291115).jpg

[12] S. Amantio di Nicolao, "Ants and aphids on a taro leaf in Manoa Valley, Honolulu," Wikimedia Commons, CC0 1.0 Universal Public Domain Dedication, Aug. 3, 2014. [Online]. Available: https://commons.wikimedia.org/wiki/File:Ants_and_aphids_on_a_taro_leaf_in_Manoa_valley,_Honolulu.jpg

[13] S. Nelson, "Mango (*Mangifera indica*) bacterial black spot caused by *Xanthomonas citri* pv. *mangiferae indicae*," Wikimedia Commons, CC0 1.0 Universal Public Domain Dedication, Apr. 25, 2018. [Online]. Available: https://commons.wikimedia.org/wiki/File:Mango_(Mangifera_indica)_Bacterial_black_spot_caused_by_Xanthomonas_citri_pv._mangiferae_indicae_(26832555887).jpg

[14] Y. Zhang, Z. Li, H. Liu, and J. Zhao, "Multimodal agents: From vision to reality," arXiv preprint arXiv:2504.04789, 2025. [Online]. Available: https://arxiv.org/abs/2504.04789

[15] H. Wang, Y. Chen, L. Zhang, and Q. Li, "MA3: A multimodal agricultural agent architecture for plant disease analysis," arXiv preprint arXiv:2509.17044, 2025. [Online]. Available: https://arxiv.org/abs/2509.17044

[16] M. Hossain, M. A. Hossain, M. Rahman, and M. S. Islam, "PDD-AGENT: Multimodal large language model-driven AI agent for enhanced plant disease diagnosis," *arXiv preprint arXiv:2402.01968*, 2024. [Online]. Available: https://arxiv.org/abs/2402.01968

[17] A. Singh, A. K. Goyal, R. Singh, and S. K. Sahu, "Artificial intelligence-based plant disease detection: A comprehensive review," *Frontiers in Plant Science*, vol. 15, Article 1356260, 2024. [Online]. Available: https://www.frontiersin.org/articles/10.3389/fpls.2024.1356260/full